

[Наименование образовательной организации]

[Факультет или институт]

[Кафедра]

ОТЧЁТ

по лабораторной работе № 3

Регрессия

по дисциплине

[Наименование дисциплины]

Выполнил: [ФИО студента]

Группа: [Номер группы]

Проверил: [ФИО преподавателя]

Подпись: _____

Дата: _____

[Город] 2026

СОДЕРЖАНИЕ

1 Цель и задачи работы	4
2 Краткие теоретические сведения	5
3 Исходные данные и среда выполнения	6
4 Подробный процесс выполнения	7
5 Данные	8
6 Методика	9
7 Метрики на тесте	9
8 Подобранные параметры	9
9 Ограничения и интерпретация	10
10 Истинные и предсказанные ставки	10
11 Остатки относительно прогноза	11
12 Распределения остатков	11
13 Коэффициенты моделей	11
14 Гистограммы — исходные данные	12
15 Диаграммы рассеяния — исходные данные	13
16 Ящики с усами: ставка по категории кредита — исходные данные	14
17 Гистограммы — очищенная тренировочная выборка	14
18 Диаграммы рассеяния — очищенная тренировочная выборка	15

19 Ящики с усами: ставка по категории кредита — очищенная тренировочная выборка	16
ЗАКЛЮЧЕНИЕ	17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	18
ПРИЛОЖЕНИЕ А	19

1 Цель и задачи работы

Получить и закрепить навыки предварительной обработки данных и применения методов регрессии для прогнозирования процентной ставки по кредиту.

Для достижения цели поставлены следующие задачи:

- Выбрать непрерывную цель и исключить её, а также исход кредита, из признаков.
- Выполнить визуализацию, удаление пропусков и дубликатов, масштабирование и кодирование.
- Обучить линейную регрессию, LASSO и Ridge, подобрать α по кросс-валидации.
- Вычислить MAE, RMSE и R^2 ; построить графики прогнозов, остатков и коэффициентов.
- Сравнить модели с базовым прогнозом среднего и сформулировать вывод.

2 Краткие теоретические сведения

Регрессия относится к обучению с учителем и прогнозирует непрерывную величину. В этой работе прогнозируется процентная ставка `loan_int_rate`. Линейная модель имеет вид $\hat{y} = \beta_0 + \sum \beta_j x_j$ и выбирает коэффициенты по ошибкам тренировочных объектов.

Обычная линейная регрессия минимизирует сумму квадратов остатков. LASSO добавляет L1-штраф $\alpha \cdot \sum |\beta_j|$ и может обнулять коэффициенты. Ridge добавляет L2-штраф $\alpha \cdot \sum \beta_j^2$ и уменьшает величину коэффициентов. При увеличении α штраф усиливается. Числовые признаки масштабируются, чтобы штраф сопоставимо действовал на коэффициенты. При OneHot-кодировании одна базовая категория исключается, чтобы избежать линейной зависимости индикаторов [3].

Для n тестовых объектов $MAE = \sum |y_i - \hat{y}_i| / n$; $RMSE = \sqrt{(\sum (y_i - \hat{y}_i)^2 / n)}$. RMSE сильнее реагирует на большие ошибки. $R^2 = 1 - \sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2$ сравнивает модель с прогнозом среднего тестовых значений. R^2 - доля объяснённой вариации, а не доля точных ответов. Поскольку цель измеряется в процентах, MAE и RMSE выражены в процентных пунктах.

Остаток равен $y_i - \hat{y}_i$. График истинных и предсказанных значений показывает близость к идеальной диагонали, а график остатков помогает обнаружить систематические ошибки. Коэффициенты отражают условную связь при фиксированных других признаках и не доказывают причинность. Параметр α выбирается внутри тренировочной кросс-валидации; итоговое сравнение проводится на отдельном тесте.

3 Исходные данные и среда выполнения

Использован Credit Risk Dataset автора Lao Tse с Kaggle, лицензия CC0 [2]. Таблица содержит 32 581 запись и 12 столбцов. В `person_emp_length` отсутствуют 895 значений, в `loan_int_rate` - 3116. После удаления строк с пропусками и 137 дубликатов осталось 28 501 наблюдение. Число удалённых строк с пропусками равно 3943, поскольку пропуски в двух столбцах могут встречаться в одной строке.

Числовые столбцы описывают возраст, доход, стаж, сумму кредита, ставку, отношение кредита к доходу и длительность кредитной истории. Категории описывают жильё, цель кредита, категорию кредита и историю дефолтов. `loan_status` - бинарный исход кредита. Исходный файл находится в `data/raw/credit_risk_dataset.csv` и сохраняется без изменений.

Расчёты выполнены средствами Python 3.12.14 и `scikit-learn` 1.9.1. `pandas` используется для таблиц, `NumPy` - для массивов, `matplotlib` и `seaborn` для графиков. Точные версии зависимостей сохранены в `requirements-lock.txt` соответствующей лабораторной. Аномалии возраста и стажа не удалялись; они учитываются как ограничение интерпретации результатов.

4 Подробный процесс выполнения

Шаг 1. Целью выбрана ставка `loan_int_rate`, поскольку она является непрерывной. Из признаков удалены `loan_int_rate` и `loan_status`. Последний характеризует исход кредита и не используется для прогноза ставки. `loan_grade` сохранён как известная категория кредита; это существенное условие интерпретации.

Шаг 2. После `dropna` и `drop_duplicates` выполняется разбиение 80/20 с `random_state=42`. В отличие от классификации, стратификация здесь не применяется. Гистограммы, ящики с усами и диаграммы рассеяния построены для исходной таблицы и отдельно для очищенной тренировочной части.

Шаг 3. `StandardScaler` и `OneHotEncoder(drop="first")` включены в Pipeline. Категории представлены индикаторами без базового уровня. Средние и масштабы вычисляются только по обучающей части каждого фолда. Преобразованные `train/test` экспортированы отдельно в `Лаба3/processed`; общие данные не перезаписаны.

Шаг 4. Линейная регрессия обучена без параметра штрафа. LASSO проверяет $\alpha=0.0001/0.001/0.01/0.1/1/10$; Ridge - $\alpha=0.01/0.1/1/10/100/1000$. `GridSearchCV` использует пять перемешанных фолдов и минимальную RMSE. `max_iter=50000` ограничивает число итераций LASSO; лучшая модель сошлась за 18 итераций.

Шаг 5. На отдельном тесте рассчитаны MAE, RMSE и R^2 . Для сравнения `DummyRegressor` прогнозирует среднюю тренировочную ставку. Сохранены графики истинных и предсказанных значений, остатки, их распределения и коэффициенты.

Шаг 6. `view_results.py` формирует отчёт и страницу результатов из CSV и JSON. Снимок фактической страницы подтверждает метрики и выбранные параметры. Программа обучения приведена целиком в приложении А.

Команда просмотра из корня проекта: `.\venv\Scripts\python Лаба3/view_results.py`. Для повторного расчёта к команде добавляется `--rebuild`.

На рисунке 1 представлен результат этапа: снимок локальной страницы с результатами лабораторной.

Лабораторная №3. Регрессия

Данные

Цель — процентная ставка `loan_int_rate`. После удаления 3943 строк с пропусками и 137 дубликатов осталось 28501 строк. Обучение: 22800, тест: 5701, `seed=42`.

Методика

Числовые признаки стандартизированы, категории кодируются OneHotEncoder с исключением базовой категории. `loan_int_rate` и `loan_status` исключены из признаков. Pipeline обучает преобразования внутри каждого из 5 фолдов KFold. Параметры выбираются по минимальной CV RMSE; тест используется только для итоговых метрик. LASSO перебирает `alpha` 0.0001/0.001/0.01/0.1/1/10, Ridge — 0.01/0.1/1/10/100/1000. У обычной линейной регрессии подбираемого штрафа нет.

Метрики

model	CV_RMSE	MAE	RMSE	R2	train_RMSE
LinearRegression	1.006876	0.784865	1.000159	0.903575	1.005613
LASSO	1.006863	0.784725	1.000096	0.903587	1.005619
Ridge	1.006875	0.784813	1.000154	0.903576	1.005614

MAE — средняя абсолютная ошибка; RMSE сильнее штрафует большие ошибки. Обе ошибки измеряются в процентных пунктах ставки: например, прогноз 11% вместо 10% даёт ошибку 1 п.п. R^2 сравнивает квадратичную ошибку модели с прогнозом среднего значения; 0.904 означает около 90.4% объяснённой тестовой вариации, а не долю точных ответов. Остаток = истинное значение – прогноз.

- LinearRegression: параметры
- LASSO: параметры
- Ridge: параметры

Вывод

Рисунок 1 - Снимок локальной страницы с результатами лабораторной

5 Данные

Цель — процентная ставка `loan_int_rate`. После удаления 3943 строк с пропусками и 137 дубликатов осталось 28501 строк. Обучение: 22800, тест: 5701, `seed=42`.

6 Методика

Числовые признаки стандартизированы, категории кодируются OneHotEncoder с исключением базовой категории. loan_int_rate и loan_status исключены из признаков. Pipeline обучает преобразования внутри каждого из 5 фолдов KFold. Параметры выбираются по минимальной CV RMSE; тест используется только для итоговых метрик. LASSO перебирает alpha 0.0001/0.001/0.01/0.1/1/10, Ridge — 0.01/0.1/1/10/100/1000. У обычной линейной регрессии подбираемого штрафа нет.

7 Метрики на тесте

В таблице 1 приведены результаты сравнения моделей.

Таблица 1 - Результаты сравнения моделей

model	CV_RMSE	MAE	RMSE	R2	train_RMS E
LinearRegression	1.006876	0.784865	1.000159	0.903575	1.005613
LASSO	1.006863	0.784725	1.000096	0.903587	1.005619
Ridge	1.006875	0.784813	1.000154	0.903576	1.005614

MAE — средняя абсолютная ошибка; RMSE сильнее штрафует большие ошибки. Обе ошибки измеряются в процентных пунктах ставки: например, прогноз 11% вместо 10% даёт ошибку 1 п.п. R² сравнивает квадратичную ошибку модели с прогнозом среднего значения; 0.904 означает около 90.4% объяснённой тестовой вариации, а не долю точных ответов. Остаток = истинное значение – прогноз.

8 Подобранные параметры

LinearRegression

Параметры: { "best_parameters": {}, "intercept": 7.30178376319885, "iterations": null }

LASSO

Параметры: { "best_parameters": { "model__alpha": 0.0001 }, "intercept": 7.304721987014661, "iterations": 18 }

Ridge

Параметры: { "best_parameters": { "model__alpha": 0.1 }, "intercept": 7.302436724561612, "iterations": null }

9 Ограничения и интерпретация

Сильный вклад имеют категории `loan_grade`: они отражают категорию кредита и тесно связаны со ставкой. Результат описывает прогноз при уже известной категории; порядок назначения категории и ставки в данных не установлен, поэтому это не доказательство возможности предсказать ставку до кредитного решения. Коэффициенты категорий сравниваются с базовой категорией, числовые — на одно стандартное отклонение. Коэффициенты не доказывают причинность. Аномальные возраст и стаж сохранены; удаление пропусков может менять состав выборки. Оценка основана на одном случайном разбиении, без проверки переноса на другой период. Полосы на графике прогнозов отражают сильное влияние `loan_grade`.

10 Истинные и предсказанные ставки

На рисунке 2 представлен результат этапа: истинные и предсказанные ставки.

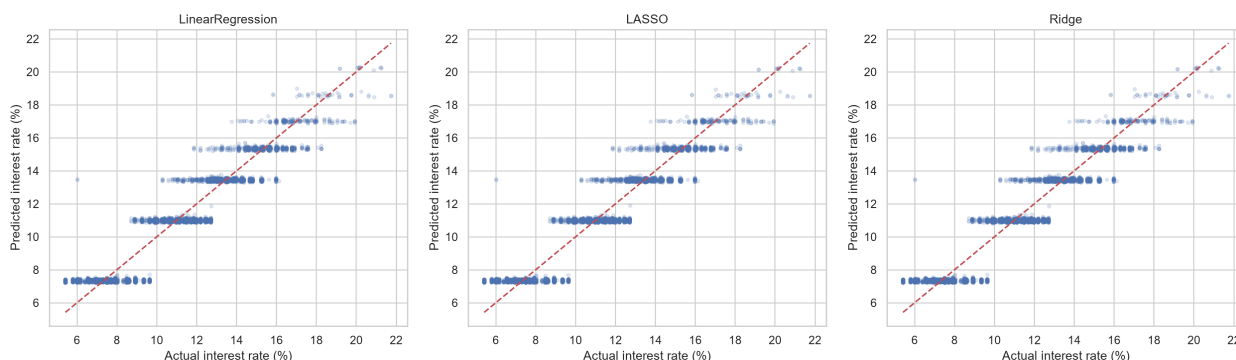


Рисунок 2 - Истинные и предсказанные ставки

11 Остатки относительно прогноза

На рисунке 3 представлен результат этапа: остатки относительно прогноза.

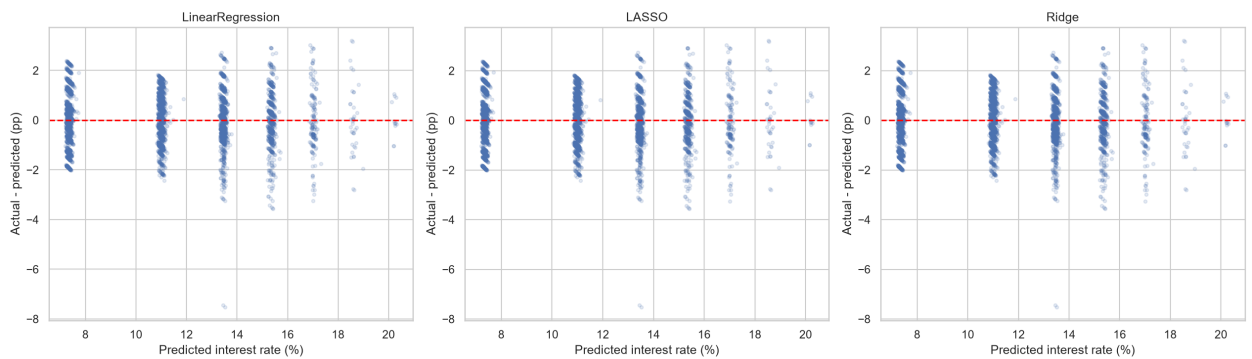


Рисунок 3 - Остатки относительно прогноза

12 Распределения остатков

На рисунке 4 представлен результат этапа: распределения остатков.

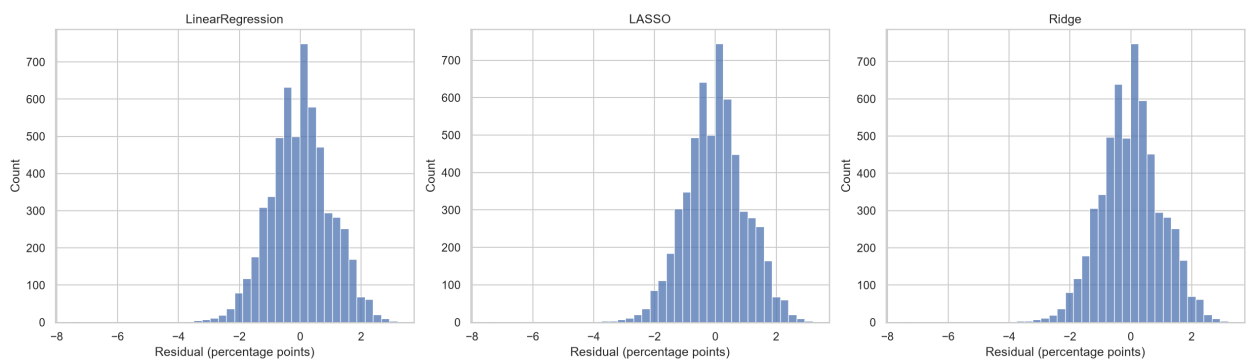


Рисунок 4 - Распределения остатков

13 Коэффициенты моделей

На рисунке 5 представлен результат этапа: коэффициенты моделей.

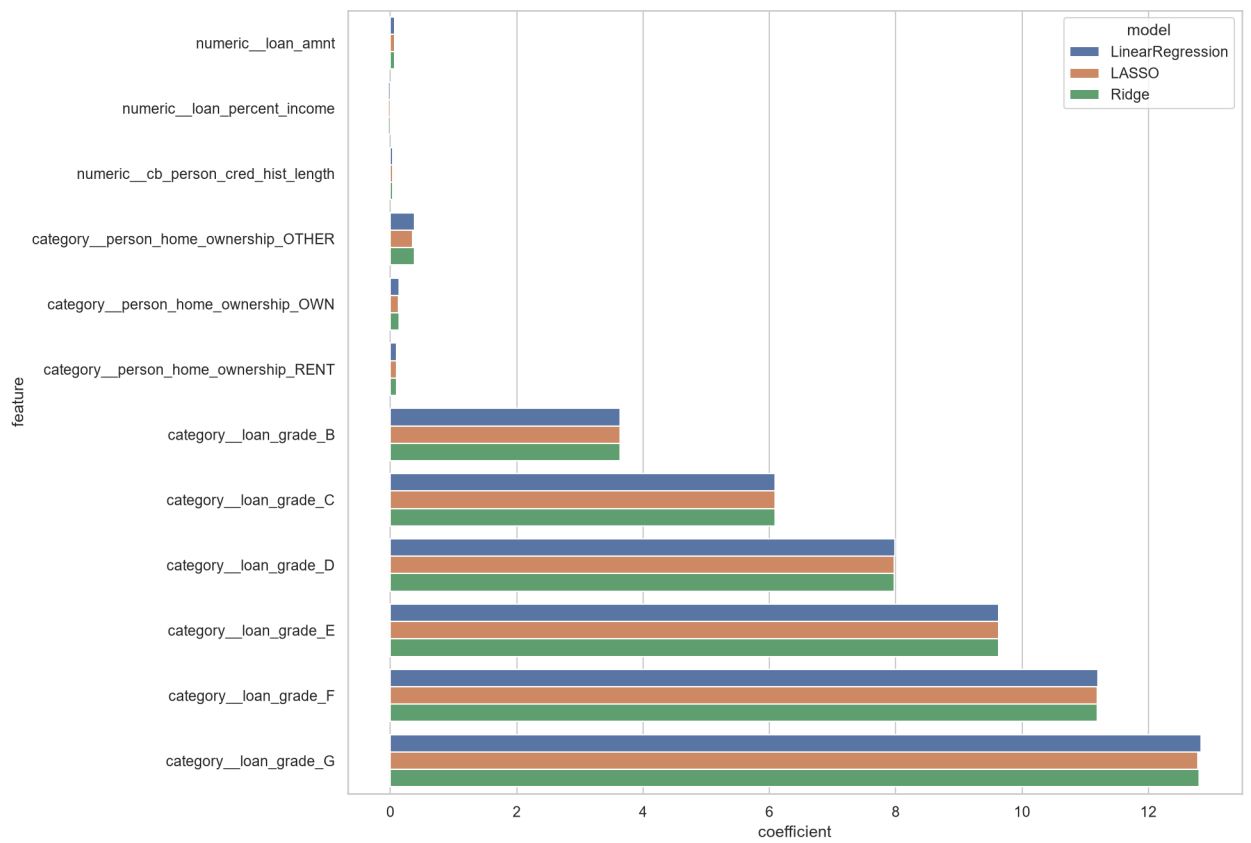


Рисунок 5 - Коэффициенты моделей

14 Гистограммы — исходные данные

На рисунке 6 представлен результат этапа: гистограммы — исходные данные.

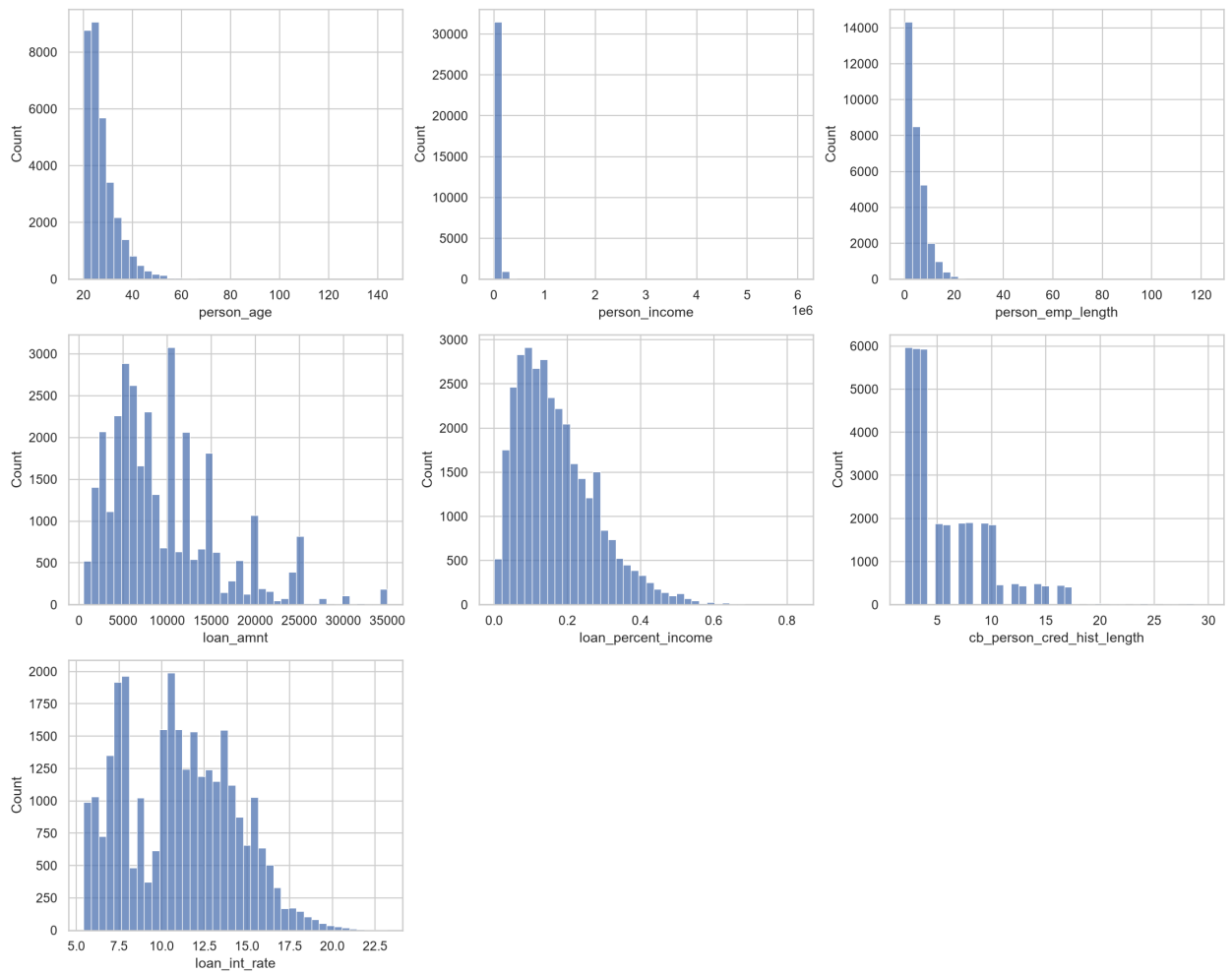


Рисунок 6 - Гистограммы — исходные данные

15 Диаграммы рассеяния — исходные данные

На рисунке 7 представлен результат этапа: диаграммы рассеяния — исходные данные.

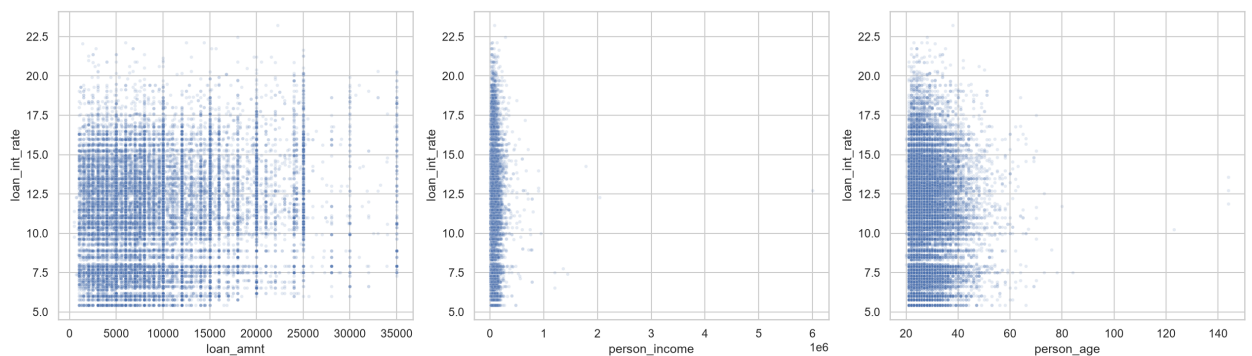
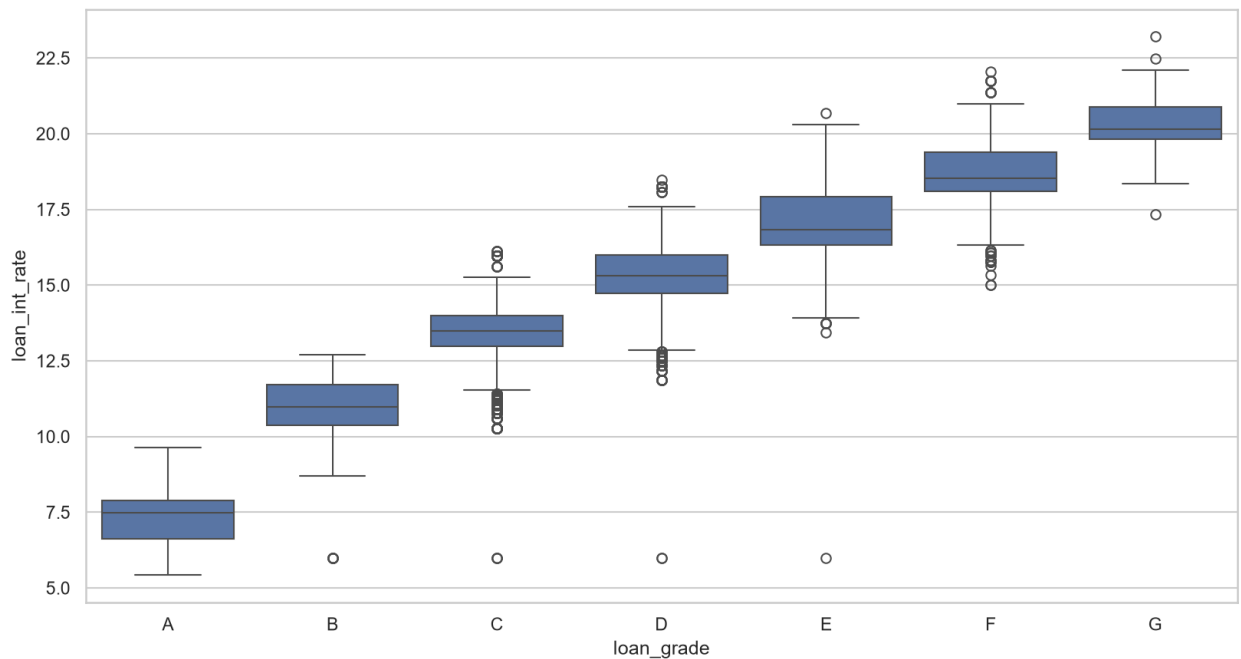


Рисунок 7 - Диаграммы рассеяния — исходные данные

16 Ящики с усами: ставка по категории кредита — исходные данные

На рисунке 8 представлен результат этапа: ящики с усами: ставка по категории кредита — исходные данные.



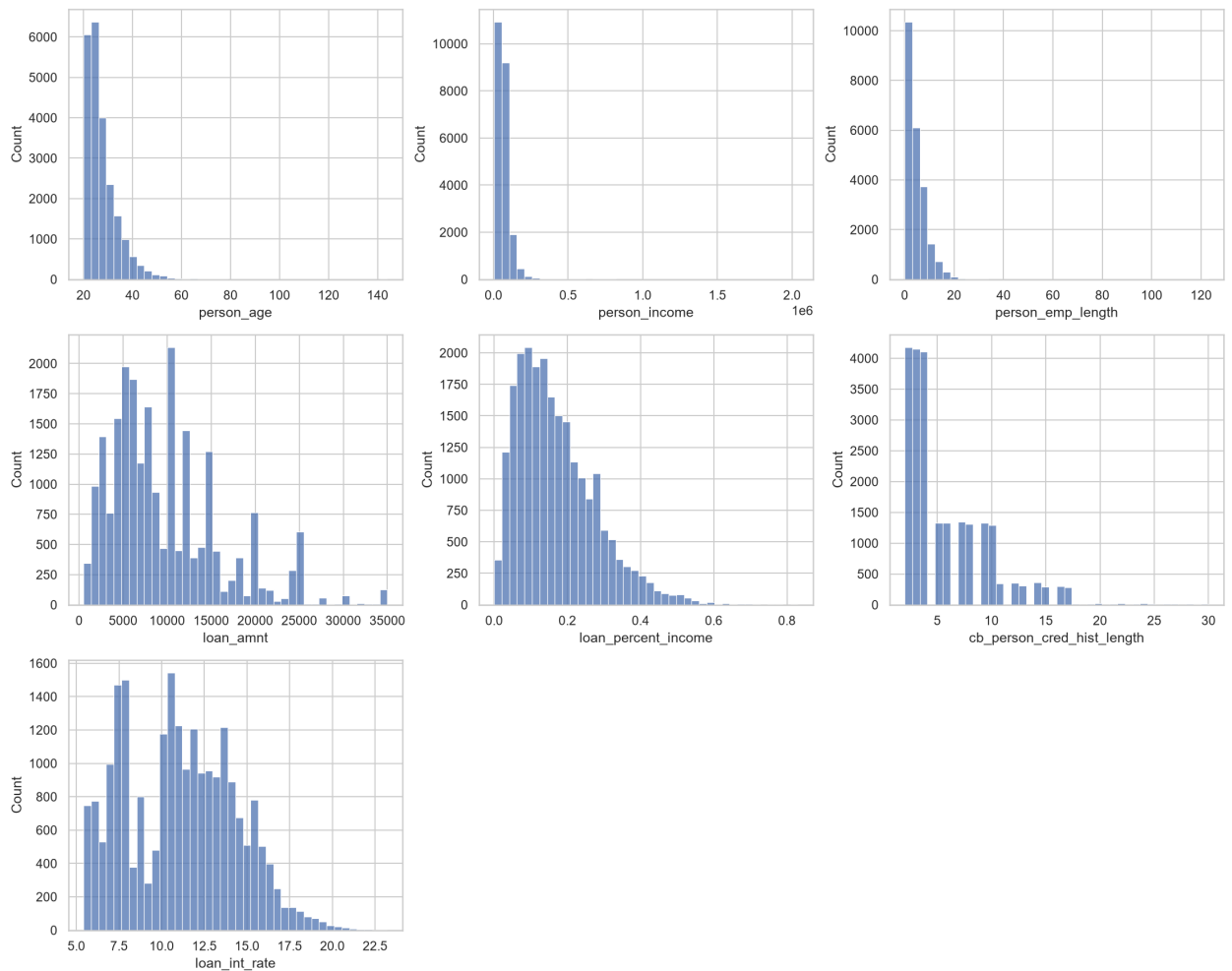


Рисунок 9 - Гистограммы — очищенная тренировочная выборка

18 Диаграммы рассеяния — очищенная тренировочная выборка

На рисунке 10 представлен результат этапа: диаграммы рассеяния — очищенная тренировочная выборка.

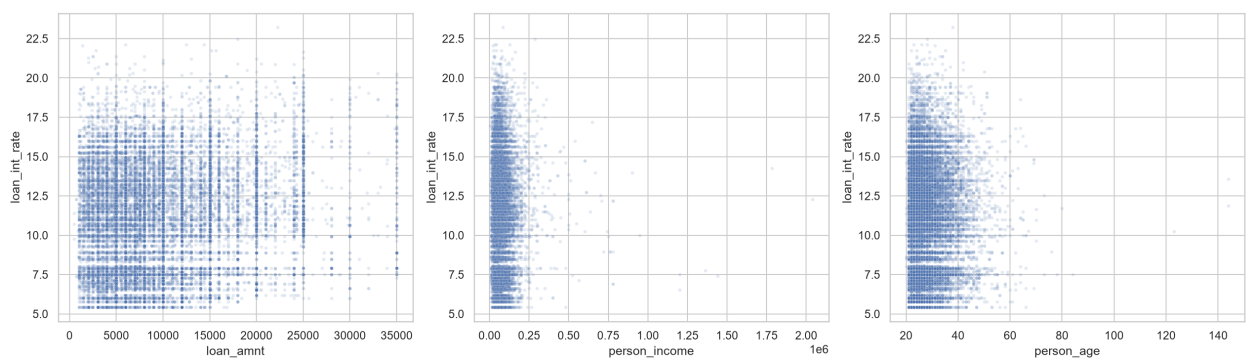


Рисунок 10 - Диаграммы рассеяния — очищенная тренировочная выборка

19 Ящики с усами: ставка по категории кредита — очищенная тренировочная выборка

На рисунке 11 представлен результат этапа: ящики с усами: ставка по категории кредита — очищенная тренировочная выборка.

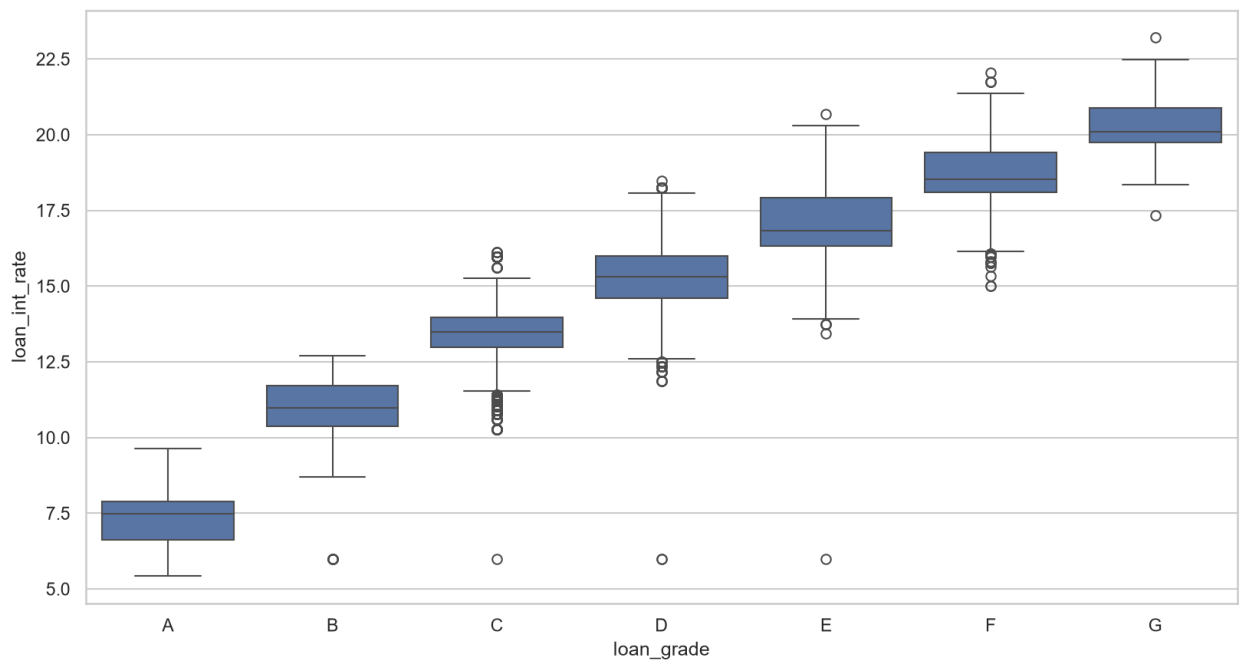


Рисунок 11 - Ящики с усами: ставка по категории кредита — очищенная тренировочная выборка

ЗАКЛЮЧЕНИЕ

Цель достигнута: выполнены очистка, визуализация и масштабирование данных, обучены линейная регрессия, LASSO и Ridge, подобраны параметры и оценены прогнозы. Все модели имеют RMSE около 1 процентного пункта и R^2 около 0.904 против RMSE 3.2215 у прогноза среднего. По CV выбрана LASSO с $\alpha=0.0001$, но различия методов пренебрежимо малы. Результат предполагает заранее известную категорию кредита `loan_grade` и не доказывает причинную связь признаков со ставкой.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32-2001. Отчёт о научно-исследовательской работе. Структура и правила оформления. URL: <https://meganorm.ru/Data2/1/4294847/4294847312.htm> (дата обращения: 03.10.2026).
2. Lao Tse. Credit Risk Dataset. Kaggle. URL: <https://www.kaggle.com/datasets/laotse/credit-risk-dataset> (дата обращения: 03.10.2026).
3. Scikit-learn. Руководство пользователя. URL: https://scikit-learn.org/stable/modules/linear_model.html (дата обращения: 03.10.2026).
4. Исходный код и сохранённые результаты проекта ОМО. URL: <https://github.com/Gemoroyy1/ОМО> (дата обращения: 03.10.2026).

ПРИЛОЖЕНИЕ А

(обязательное)

Программный код regression.py

Приведён полный исходный текст файла Лаба3/regression.py. Длинные строки перенесены только для печати; исходный файл в репозитории сохраняет исходные переносы и отступы. Код выполнялся в среде Python проекта.

```
"""Лабораторная №3: прогноз процентной ставки. python
    Лаба3/regression.py."""

import os
from pathlib import Path
import json
import platform

ROOT = Path(__file__).resolve().parent
os.environ.setdefault("MPLCONFIGDIR", str(ROOT /
    ".cache/matplotlib"))
os.environ.setdefault("OMP_NUM_THREADS", "2")
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import sklearn
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression, Lasso, Ridge
from sklearn.metrics import mean_absolute_error,
    mean_squared_error, r2_score
from sklearn.model_selection import GridSearchCV, KFold,
    train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

TARGET = "loan_int_rate"
NUMERIC = ["person_age", "person_income", "person_emp_length",
    "loan_amnt",
    "loan_percent_income", "cb_person_cred_hist_length"]
OUTPUT = ROOT / "reports"
```

```

def save(name):
    plt.tight_layout()
    plt.savefig(OUTPUT / name, dpi=150, bbox_inches="tight")
    plt.close()

def make_pipeline(model, categories):
    transform = ColumnTransformer([
        ("numeric", StandardScaler(), NUMERIC),
        # Базовая категория исключена: устраняем зависимость
        # dummy-столбцов.
        ("category", OneHotEncoder(drop="first",
            handle_unknown="ignore",
            sparse_output=False),
        categories),
    ])
    return Pipeline([("preprocess", transform), ("model",
        model)])

def visualize(data, stage):
    selected = NUMERIC + [TARGET]
    fig, axes = plt.subplots(3, 3, figsize=(15, 12))
    for ax, feature in zip(axes.flat, selected):
        sns.histplot(data=data, x=feature, bins=40, ax=ax)
    for ax in list(axes.flat)[len(selected):]:
        ax.set_visible(False)
    save(f"{stage}_histograms.png")
    fig, axes = plt.subplots(1, 3, figsize=(17, 5))
    for ax, feature in zip(axes, ["loan_amnt", "person_income",
        "person_age"]):
        sns.scatterplot(data=data, x=feature, y=TARGET,
            alpha=0.15, s=10, ax=ax)
    save(f"{stage}_scatterplots.png")
    plt.figure(figsize=(11, 6))
    sns.boxplot(data=data, x="loan_grade", y=TARGET,
        order=list("ABCDEFGG"))
    save(f"{stage}_boxplots.png")

def evaluate(actual, predicted):
    return {"MAE": float(mean_absolute_error(actual,
        predicted)),
        "RMSE": float(np.sqrt(mean_squared_error(actual,
        predicted))),
        "R2": float(r2_score(actual, predicted))}

```

```

def main():
    OUTPUT.mkdir(parents=True, exist_ok=True)
    processed = ROOT / "processed"
    processed.mkdir(parents=True, exist_ok=True)
    sns.set_theme(style="whitegrid")
    raw = pd.read_csv(ROOT.parent /
        "data/raw/credit_risk_dataset.csv")
    complete = raw.dropna()
    clean = complete.drop_duplicates()
    # loan_status — исход кредита, его нельзя использовать для
    прогноза ставки.
    x = clean.drop(columns=[TARGET, "loan_status"])
    y = clean[TARGET]
    x_train, x_test, y_train, y_test = train_test_split(x, y,
        test_size=0.2, random_state=42)
    # Исследуем признаки только тренировочной части: тест
    остаётся для итоговой оценки.
    visualize(clean.loc[x_train.index], "train_clean")
    # Для иллюстрации очистки до обучения — весь исходный набор
    отдельно.
    # Эти графики не используются для подбора параметров.
    visualize(raw, "raw")
    categories = [name for name in x if name not in NUMERIC]
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    models = {
        "LinearRegression": (LinearRegression(), {}),
        "LASSO": (Lasso(max_iter=50000), {"model__alpha":
            [0.0001, 0.001, 0.01, 0.1, 1, 10]}),
        "Ridge": (Ridge(), {"model__alpha": [0.01, 0.1, 1, 10,
            100, 1000]}),
    }
    rows, details, coefficients = [], {}, []
    predictions = pd.DataFrame({"source_row": x_test.index,
        "actual": y_test.to_numpy()})
    fig_predictions, axes_predictions = plt.subplots(1, 3,
        figsize=(17, 5))
    fig_residuals, axes_residuals = plt.subplots(1, 3,
        figsize=(17, 5))
    fig_hist, axes_hist = plt.subplots(1, 3, figsize=(17, 5))
    baseline = DummyRegressor(strategy="mean").fit(x_train,
        y_train)
    baseline_metrics = evaluate(y_test,
        baseline.predict(x_test))
    final_pipeline = None
    for ax_p, ax_r, ax_h, (name, (model, grid)) in zip(
        axes_predictions, axes_residuals, axes_hist,
        models.items()):

```

```

        print(f"Training {name}...", flush=True)
        search = GridSearchCV(make_pipeline(model, categories),
                                grid, cv=cv,

scoring="neg_root_mean_squared_error", n_jobs=2,
error_score="raise")
        search.fit(x_train, y_train)
        best = search.best_estimator_
        predicted = best.predict(x_test)
        residual = y_test.to_numpy() - predicted
        row = {"model": name, "CV_RMSE":
float(-search.best_score_), **evaluate(y_test, predicted)}
        row["train_RMSE"] = evaluate(y_train,
best.predict(x_train))["RMSE"]
        rows.append(row)
        predictions[name] = predicted
        details[name] = {"best_parameters": search.best_params_,
                        "intercept":
float(best.named_steps["model"].intercept_),
                        "iterations":
int(best.named_steps["model"].n_iter_) if name == "LASSO"
else None}
        pd.DataFrame(search.cv_results_).to_csv(OUTPUT /
f"{name}_cv.csv", index=False)
        coefficients.append(pd.DataFrame({"model": name,
                        "feature":
best.named_steps["preprocess"].get_feature_names_out(),
                        "coefficient": best.named_steps["model"].coef_}))
        ax_p.scatter(y_test, predicted, alpha=0.15, s=10)
        bounds = [min(y_test.min(), predicted.min()),
max(y_test.max(), predicted.max())]
        ax_p.plot(bounds, bounds, "r--")
        ax_p.set(xlabel="Actual interest rate (%)",
ylabel="Predicted interest rate (%)", title=name)
        ax_r.scatter(predicted, residual, alpha=0.15, s=10)
        ax_r.axhline(0, color="red", linestyle="--")
        ax_r.set(xlabel="Predicted interest rate (%)",
ylabel="Actual - predicted (pp)", title=name)
        sns.histplot(residual, bins=40, ax=ax_h)
        ax_h.set(xlabel="Residual (percentage points)",
title=name)
        final_pipeline = best
        print(row, flush=True)
for fig, name in [(fig_predictions, "actual_predicted.png"),
                  (fig_residuals, "residuals.png"),
                  (fig_hist, "residual_histograms.png")]:
        plt.figure(fig.number)
        save(name)

```

```

coef = pd.concat(coefficients, ignore_index=True)
coef.to_csv(OUTPUT / "coefficients.csv", index=False)
top = coef.groupby("feature").coefficient.apply(lambda s:
s.abs().max()).nlargest(12).index
plt.figure(figsize=(13, 9))
sns.barplot(data=coef[coef.feature.isin(top)],
x="coefficient", y="feature", hue="model")
save("coefficients.png")
metrics = pd.DataFrame(rows)
metrics.to_csv(OUTPUT / "metrics.csv", index=False)
predictions.to_csv(OUTPUT / "predictions.csv", index=False)
# Экспорт стандартизированных признаков с параметрами только
тренировочной выборки.
prep = final_pipeline.named_steps["preprocess"]
for name, subset, target in [("train", x_train, y_train),
("test", x_test, y_test)]:
    transformed = pd.DataFrame(prepare.transform(subset),
columns=prep.get_feature_names_out())
    transformed.insert(0, "source_row", subset.index)
    transformed[TARGET] = target.to_numpy()
    transformed.to_csv(processed /
f"{name}_transformed.csv", index=False)
scaler = prep.named_transformers_["numeric"]
winner = metrics.loc[metrics.CV_RMSE.idxmin(), "model"]
details["experiment"] = {
    "target": TARGET, "excluded": ["loan_status"],
"original_rows": len(raw),
    "removed_missing": len(raw)-len(complete),
"removed_duplicates": len(complete)-len(clean),
    "clean_rows": len(clean), "train_rows": len(x_train),
"test_rows": len(x_test),
    "random_state": 42, "selected_by_cv_rmse": winner,
"baseline": baseline_metrics,
    "numeric_mean": dict(zip(NUMERIC,
scaler.mean_.tolist())),
    "numeric_scale": dict(zip(NUMERIC,
scaler.scale_.tolist())),
    "python": platform.python_version(), "sklearn":
sklearn.__version__,
    "lasso_zero_coefficients": int((coef.loc[coef.model ==
"LASSO", "coefficient"] == 0).sum()),
}
(OUTPUT / "results.json").write_text(json.dumps(details,
ensure_ascii=False, indent=2), encoding="utf-8")
print(metrics.to_string(index=False))
print("Selected by CV RMSE:", winner)

```

```
if __name__ == "__main__":  
    main()
```